Name : Vaishnavi Eknath Avhad
Class : D20C
Roll No. : 2

Practical 6

Aim : Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory

Importance of require Statements in Solidity

In Solidity, the require statement is mainly used to check whether a certain condition is true before continuing with a function. It works like a safety check and makes sure that only valid inputs or authorized users can perform specific actions.

If the condition inside require is false, the function stops immediately. Any changes made to the contract during that transaction are also reverted. This helps prevent incorrect data from being stored on the blockchain.

For example, in a voting smart contract, require can be used to check :

- Whether the caller has permission to vote, such as require(voters[msg.sender].weight > 0, "Has no right to vote");
- Whether the voter has already voted.
- Whether the person calling the function is the chairperson before giving voting rights.

Therefore, require statements are important for maintaining the security and correctness of smart contracts. They can also include error messages, which makes it easier to understand why a transaction failed.

Mapping

A mapping in Solidity is a data structure that connects a key with a particular value. It is somewhat similar to a hash table in other programming languages.

Its general syntax is :

mapping(keyType => valueType)

For example :

mapping(address => Voter) public voters;

In this example, every Ethereum address is associated with a Voter structure. This is useful in a voting contract because information about each voter, such as whether they have voted or their voting weight, can be stored against their address.
Mappings are efficient for finding data using a key. However, they do not have a length property and cannot be directly iterated through like an array.

Storage

Storage is the permanent data area of a Solidity smart contract. Data stored in storage remains on the blockchain even after a function has finished executing.

Variables declared at the contract level are generally stored in storage. For example, voter information stored in the voters mapping can be accessed later as long as it has not been changed or deleted.
Storage operations are relatively expensive because they involve modifying blockchain data and therefore consume gas.

Memory

Memory is temporary storage that is used while a function is running. Unlike storage, data kept in memory is not permanently saved on the blockchain. Once the function finishes, the memory used by it is released.

Memory is commonly used for temporary variables, function parameters, and calculations that do not need to be stored permanently.
For example, when working with proposal names or creating temporary values inside a function, memory can be used instead of storage. Since memory is temporary, it is generally less expensive than writing data permanently to storage.
Therefore, understanding the difference between storage and memory is important when developing Solidity contracts because it can affect both the performance and gas cost of a contract.

Why Use bytes32 Instead of string?

In some earlier versions of the Ballot contract, bytes32 was used to store proposal names instead of string. One reason for this was efficiency.

bytes32 is a fixed-size data type that stores exactly 32 bytes. Because its size is fixed, it can be handled efficiently by the Ethereum Virtual Machine (EVM). However, the main limitation is that it can only hold a maximum of 32 bytes, so it is less suitable for longer or more flexible text.
On the other hand, string is a dynamically sized data type. It can store text of different lengths, which makes it more convenient when dealing with proposal names. However, working with dynamic strings can require more storage and processing.
For example, a proposal name can be stored as a bytes32 value when efficiency is important, while a string can be used when readability and flexibility are more important.
In simple terms, bytes32 is useful when a fixed-size value and efficient storage are preferred, whereas string is more convenient when working with normal text. The choice between them depends on the requirements of the smart contract.

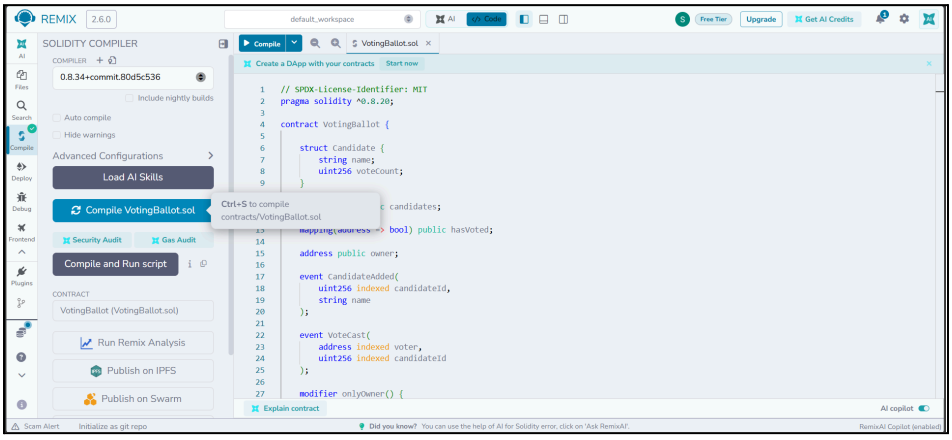
Code and Output

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract VotingBallot {
    struct Candidate {
        string name;
        uint256 voteCount;
    }
    Candidate[] public candidates;
    mapping(address => bool) public hasVoted;
    address public owner;
    event CandidateAdded(
        uint256 indexed candidateId,
        string name
    );
    event VoteCast(
        address indexed voter,
        uint256 indexed candidateId
    );
    modifier onlyOwner() {
        require(
            msg.sender == owner,
            "Only owner can perform this action"
        );
        _;
    }
    constructor() {
        owner = msg.sender;
    }
    function addCandidate(
        string memory _name
    ) public onlyOwner {
        require(
            bytes(_name).length > 0,
            "Candidate name cannot be empty"
        );
        candidates.push(
            Candidate({
                name: _name,
                voteCount: 0
            })
        );
        emit CandidateAdded(
            candidates.length - 1,
            _name
        );
    }
    function vote(
        uint256 _candidateId
    ) public {
        require(
            !hasVoted[msg.sender],
            "You have already voted"
        );
        require(
            _candidateId < candidates.length,
            "Invalid candidate"
        );
        hasVoted[msg.sender] = true;
        candidates[_candidateId].voteCount++;
        emit VoteCast(
            msg.sender,
            _candidateId
        );
    }
    function getCandidateCount()
        public
        view
        returns (uint256)
    {
        return candidates.length;
    }

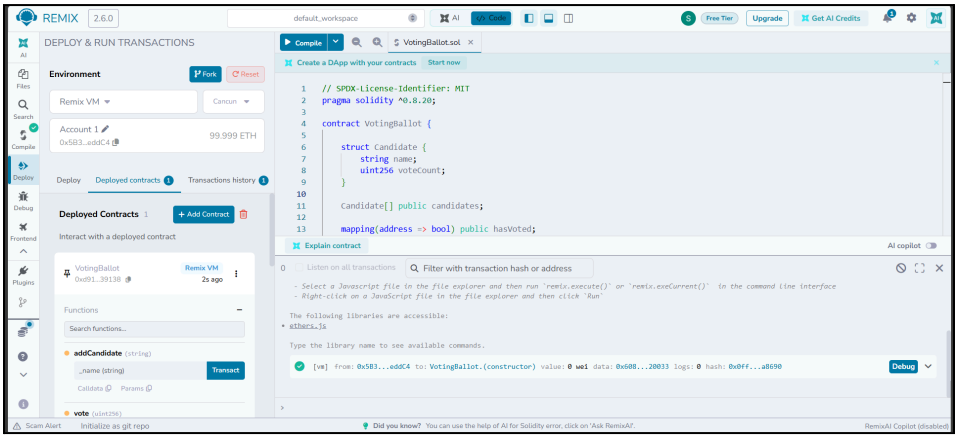
    function getCandidate(
        uint256 _candidateId
    )
        public
        view
        returns (
            string memory,
            uint256
        )
    {
        require(
            _candidateId < candidates.length,
            "Invalid candidate"
        );
        Candidate memory candidate =
            candidates[_candidateId];
        return (
            candidate.name,
```

```
            candidate.voteCount
        );
    }
    function getWinner()
        public
        view
        returns (
            string memory winnerName,
            uint256 winnerVotes
        )
    {
        require(
            candidates.length > 0,
            "No candidates available"
        );
        uint256 winningIndex = 0;
        for (
            uint256 i = 1;
            i < candidates.length;
            i++
        ) {
            if (
                candidates[i].voteCount >
                candidates[winningIndex].voteCount
            ) {
                winningIndex = i;
            }
        }
        winnerName =
            candidates[winningIndex].name;
        winnerVotes =
            candidates[winningIndex].voteCount;
    }
}
```

Contract Compiled :



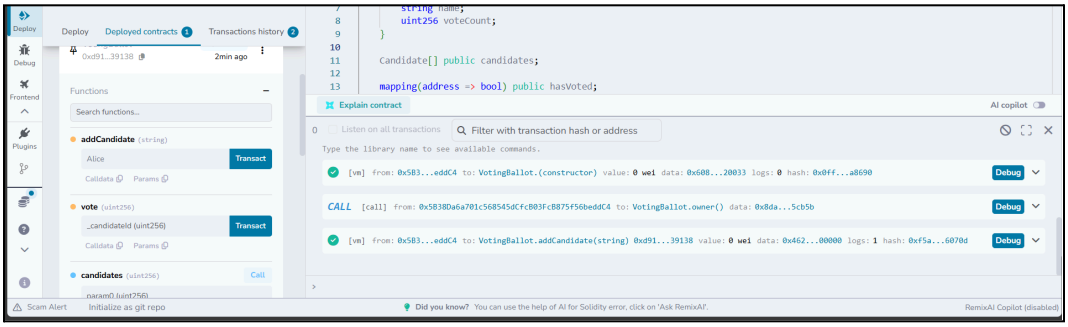
Contract Deployed :



Owner of contract :



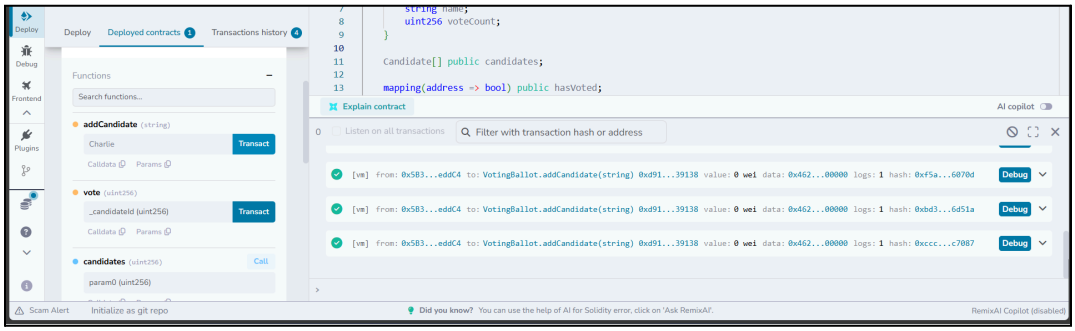
Added Alice as candidate :



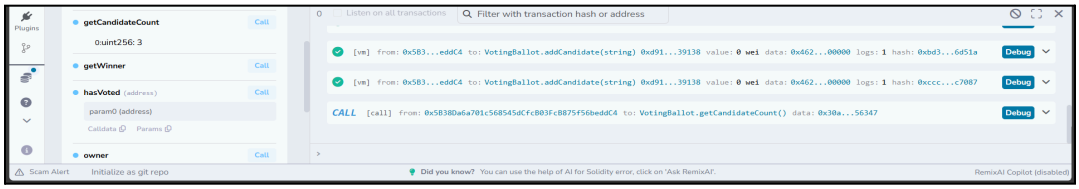
Added 2nd Candidate Bob :



Added 3rd candidate Charlie :



Calling getCandidateCount :



The output is right as its 3.

Calling getCandiate function on Alice :



Alice has 0 votes.

Voting for Alice using Account 2 :



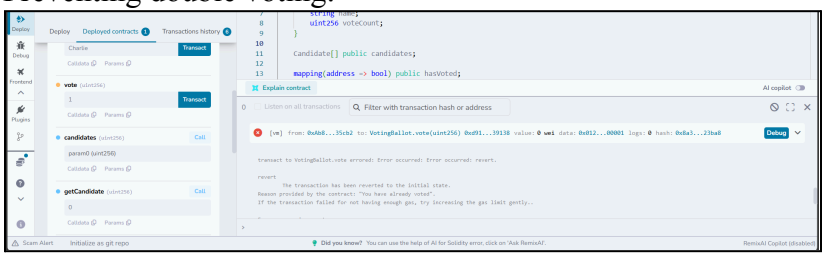
Verify that the vote is given to Alice :



Verify Account 2 has voted :

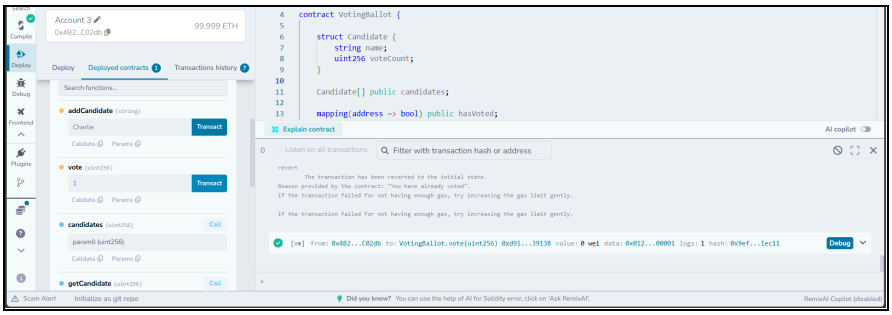


Preventing double voting:



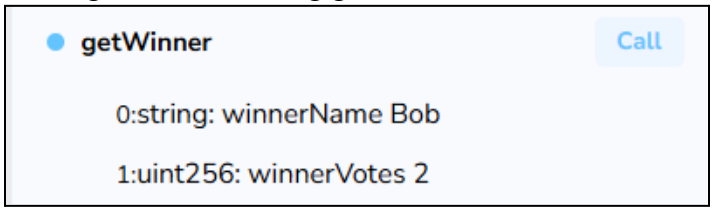
Double voting prevented successfully

Using 3rd and 4th account to vote for Bob(1) :





Finding the winner using getWinner :



Conclusion

In this experiment, a Voting/Ballot smart contract was created and deployed using Solidity through the Remix IDE. We studied the use of require statements, mappings, and data location specifiers such as storage and memory to understand how they help improve the security, efficiency, and proper functioning of smart contracts. We also compared bytes32 and string for storing proposal names and learned about the difference between gas efficiency and readability. Overall, this experiment helped in understanding the basic concepts involved in developing and deploying a voting smart contract on the blockchain.